Section 7: Sliding Window

This section is part of the 5-hour LeetCode DSA Foundations course by Mukul Raina

- YouTube Course: https://youtu.be/YJMBv4Q2Tg8?si=Q1vDzusUJf753GG3
- YouTube Channel: https://www.youtube.com/@mukulraina
- Instructor LinkedIn: https://www.linkedin.com/in/mukulraina/

SLIDE 61: Section Intro, Sliding Window

SLIDE 62: The Sliding Window Mental Model

SLIDE 63: Fixed-Size Windows

SLIDE 64: Variable-Size Windows

SLIDE 65: The Shrink-and-Expand Pattern: Longest vs Shortest

SLIDE 66: Worked Example: Minimum Window Substring

SLIDE 66B: Minimum Window Substring Step-by-Step Trace

SLIDE 67: Worked Example: Longest Substring Without Repeating Characters (Revisited)

SLIDE 68: Wrong vs Right: Sliding Window Mistakes

SLIDE 69: Interview Practice: Sliding Window

SLIDE 70: Section Cheat Sheet

SLIDE 61: Section Intro, Sliding Window

Section 7: Sliding Window

When a problem asks you to find the longest, shortest, or optimal contiguous subarray or substring that satisfies some condition, and the brute force involves checking every possible subarray, sliding window is almost always the O(n) path to the answer. It is one of the highest-frequency interview patterns across Google, Meta, Amazon, and Microsoft.

The idea: instead of recomputing the answer from scratch for every possible subarray, maintain a “window” over the data and update it incrementally. Expanding the window adds one element. Shrinking the window removes one element. Each update costs O(1), and the window slides across the entire input in O(n) total.

What We’ll Cover

1. The sliding window mental model and incremental computation
2. Fixed-size window template and worked example
3. Variable-size window template and the amortized O(n) argument
4. Longest vs shortest vs count: choosing the right template variant
5. Worked examples: Minimum Window Substring, Longest Substring Without Repeating Characters

Where This Section Connects

Section	Connection
Arrays (Section 3)	Subarray patterns introduced prefix sums and Kadane's. Sliding window is the third core subarray technique.
Strings (Section 4)	"Substring" in a string problem IS "subarray" on the character array. Same templates apply directly.
Hash Maps (Section 5)	Sliding window frequently uses HashMap or HashSet to track window contents.
Two Pointers (Section 6)	Sliding window is a specialized form of same-direction two pointers, where left and right define a window with tracked state.

If you understood the two-pointer idea from Section 6, and the frequency tracking from Section 5, sliding window is where they combine into one of the most powerful interview techniques.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 62: The Sliding Window Mental Model

Why Brute Force Is Wasteful

Consider finding the maximum sum of any 3 consecutive elements in [2, 1, 5, 1, 3, 2]. The brute force computes each window independently:

```
Window [2, 1, 5]: sum = 2 + 1 + 5 = 8
Window [1, 5, 1]: sum = 1 + 5 + 1 = 7
Window [5, 1, 3]: sum = 5 + 1 + 3 = 9
Window [1, 3, 2]: sum = 1 + 3 + 2 = 6
```

Each window recomputes the sum from scratch: K additions per window, n windows. Total: O(n * K).

But notice: between consecutive windows, only two things change. One element leaves (the leftmost) and one element enters (the new rightmost). The rest of the window is identical. If we subtract the leaving element and add the entering element, each slide costs O(1) instead of O(K).

The Incremental Update

Step	Leaving	Entering	Calculation	Window Sum
Initial [2,1,5]	-	-	2 + 1 + 5	8
Slide to [1,5,1]	2	1	8 - 2 + 1	7
Slide to [5,1,3]	1	3	7 - 1 + 3	9
Slide to [1,3,2]	5	2	9 - 5 + 2	6

O(K) to initialize, O(1) per slide, O(n) total. This is the core idea.

What Can Be Tracked Incrementally

Not all properties can be maintained with O(1) updates, but many common ones can:

Window Property	Add Element	Remove Element	Data Structure
Sum	Add value	Subtract value	Single variable
Character frequency	Increment count	Decrement count	int[26] or HashMap
Set membership	Add to set	Remove from set	HashSet
Count of distinct elements	Track frequencies, increment distinct count when freq goes 0 to 1	Decrement distinct count when freq goes 1 to 0	HashMap
Maximum in window	Requires special structure	Requires special structure	Monotonic deque (Section 10)

If the property you need to track cannot be updated in O(1) per slide, sliding window may not apply, or it may need an auxiliary structure like the monotonic deque covered in Section 10.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 63: Fixed-Size Windows

When the Window Width Is Given

If the problem specifies a fixed window size K, the approach is straightforward: initialize the window over the first K elements, then slide one position at a time, adding the new element and removing the old one.

Template Applied: Maximum Sum of Subarray of Size K

Given an array of integers and a positive integer K, find the maximum sum of any contiguous subarray of size K.

```
public int maxSumSubarray(int[] nums, int k) {
    int windowSum = 0;

    for (int i = 0; i < k; i++) {
        windowSum += nums[i];
    }

    int maxSum = windowSum;

    for (int i = k; i < nums.length; i++) {
        windowSum += nums[i] - nums[i - k];
        maxSum = Math.max(maxSum, windowSum);
    }

    return maxSum;
}
```

The first loop builds the initial window of size K. The second loop slides the window: nums[i] is the new element entering on the right, nums[i - k] is the old element leaving on the left. The difference between this and brute force: one addition and one subtraction per slide instead of K additions.

Trace through [2, 1, 5, 1, 3, 2] with K = 3:

i	Entering	Leaving	windowSum	maxSum
0-2 (init)	-	-	8	8
3	nums[3]=1	nums[0]=2	8 - 2 + 1 = 7	8
4	nums[4]=3	nums[1]=1	7 - 1 + 3 = 9	9
5	nums[5]=2	nums[2]=5	9 - 5 + 2 = 6	9

Maximum sum: 9, from subarray [5, 1, 3]. Time O(n), space O(1).

When to Recognize Fixed-Size Windows

Problem statement signals: "subarray of size K", "K consecutive elements", "window of fixed length", or any problem where the subarray length is given as a parameter. If the window size is fixed and the tracked property updates in O(1), this template applies directly.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 64: Variable-Size Windows

When the Window Size Is Not Given

Many problems ask for the longest or shortest subarray that satisfies some condition, without specifying the size. Here, the window expands and contracts dynamically: the right pointer moves forward to explore, and the left pointer advances to restore a violated condition.

Template Applied: Longest Subarray with Sum at Most Target

Given an array of positive integers and a target, find the length of the longest contiguous subarray whose sum is less than or equal to the target.

```
public int longestSubarray(int[] nums, int target) {
    int left = 0;
    int windowSum = 0;
    int maxLen = 0;

    for (int right = 0; right < nums.length; right++) {
        windowSum += nums[right];

        while (windowSum > target) {
            windowSum -= nums[left];
            left++;
        }

        maxLen = Math.max(maxLen, right - left + 1);
    }

    return maxLen;
}
```

The outer for loop expands the window rightward by one element each iteration. When the window violates the condition (sum exceeds target), the inner while loop shrinks from the left until the condition is restored. After shrinking, the window is valid and as large as possible, so we update the result.

The Amortized O(n) Argument

Candidates frequently misjudge this as O(n^2) because of the nested while loop. Here is why it is O(n):

The left pointer only moves forward. Across the entire execution, left moves from 0 to at most n. The right pointer also moves from 0 to n. Each pointer contributes at most n moves. Total work: at most 2n operations = O(n).

The inner while loop does NOT run n times per outer iteration. It runs a variable number of times per iteration, but the total across ALL iterations is bounded by n. This is the amortized argument, and it is the same argument that makes many sliding window solutions O(n).

Interview Tip: When presenting a sliding window solution, proactively explain the amortized O(n) argument. Saying “the left pointer moves at most n times total across all iterations, so the total work is O(n)” demonstrates a deeper understanding than just saying “it’s O(n) because there’s one for loop.”

When to Recognize Variable-Size Windows

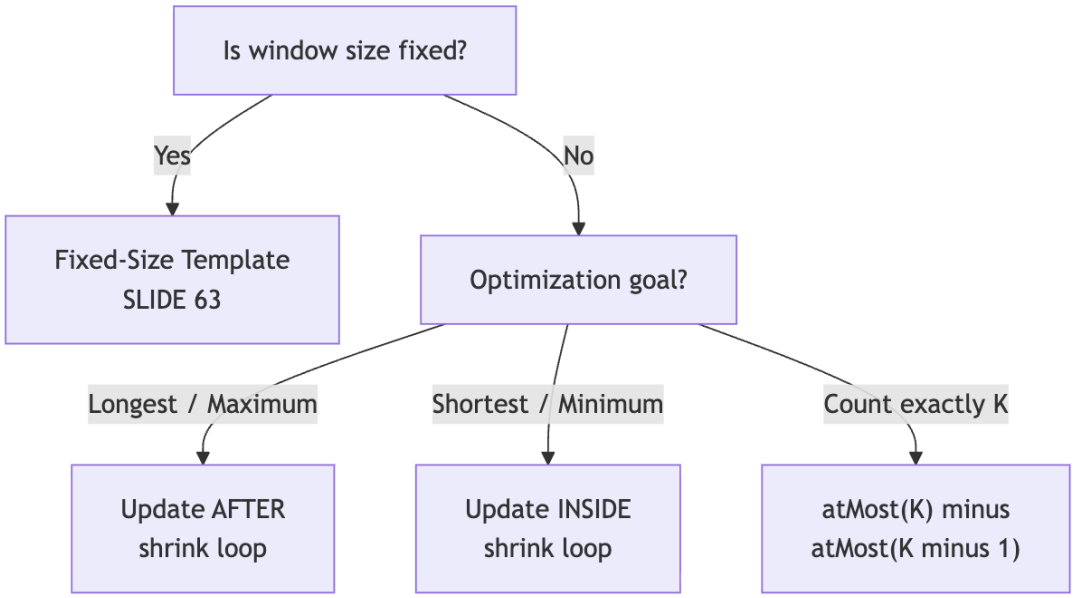
Problem statement signals: “longest subarray with condition”, “shortest subarray with condition”, “smallest window containing X”, or any optimization over contiguous elements where the window size is part of what you are finding.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 65: The Shrink-and-Expand Pattern: Longest vs Shortest

Variable-size sliding window problems fall into three categories, and each category updates the result at a different point in the code. Using the wrong update location produces subtly incorrect answers.



Category 1: Longest / Maximum (most common)

You want the largest valid window. After the shrink loop restores validity, the window is valid and as large as possible, so update the result there.

```
for (int right = 0; right < n; right++) {
    // expand: add nums[right] to window state

    while (windowViolatesCondition) {
        // shrink: remove nums[left] from window state
        left++;
    }

    maxLen = Math.max(maxLen, right - left + 1);
}
```

The result update sits AFTER the while loop. The window is valid at this point, and we want the longest valid window, so we check here.

Category 2: Shortest / Minimum

You want the smallest valid window. The window becomes valid at some point, and you want to keep shrinking to find the minimum size that still satisfies the condition. Update the result INSIDE the shrink loop.

```
for (int right = 0; right < n; right++) {
    // expand: add nums[right] to window state

    while (windowSatisfiesCondition) {
        minLen = Math.min(minLen, right - left + 1);
        // shrink: remove nums[left] from window state
```

```
            left++;
        }
    }
```

The result update sits INSIDE the while loop. Each time through the loop, the window is valid, and we record its size before shrinking further. The while loop continues as long as the window stays valid, finding progressively smaller valid windows.

Notice the structural difference: for longest, the while condition checks when the window is INVALID (shrink to restore). For shortest, the while condition checks when the window is VALID (shrink to minimize).

Category 3: Count Subarrays with Exactly K

You want to count subarrays satisfying a condition with exactly K of something. Direct counting is hard because expanding the window can both add and remove valid subarrays. The trick:

exactlyK = atMost(K) - atMost(K - 1)

Write a helper function that counts subarrays with at most K of the condition (this is a standard longest-variant sliding window). Call it twice and subtract. This decomposition converts a hard counting problem into two applications of the standard template.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 66: Worked Example: Minimum Window Substring

Problem: Minimum Window Substring (LeetCode 76)

Given two strings s and t, find the shortest substring of s that contains every character of t (including duplicates). If no such substring exists, return an empty string.

Step 1: Understand

s = “ADOBECODEBANC”, t = “ABC”. We need the shortest window in s that contains at least one A, one B, and one C. Characters in s that are not in t can appear in the window but do not count toward the requirement.

Step 2: Identify the Pattern

“Shortest substring containing all characters” maps directly to variable-size sliding window, shortest variant (update INSIDE the shrink loop, from Slide 65).

Step 3: Plan

Track what we need (character frequencies of t) and what the window has (character frequencies of the current window). Expand right to add characters. When the window contains all required characters, record the window size and shrink from the left to try to find a shorter valid window.

Two counters make this efficient: “required” is the number of distinct characters needed (size of the need map), and “have” tracks how many of those distinct characters are currently satisfied in the window. When have equals required, the window is valid.

Full Java Solution

```
public String minWindow(String s, String t) {
    if (s.length() < t.length()) return "";
```

```
    Map<Character, Integer> need = new HashMap<>();
    for (char c : t.toCharArray()) {
        need.put(c, need.getOrDefault(c, 0) + 1);
    }

    Map<Character, Integer> window = new HashMap<>();
    int have = 0;
    int required = need.size();
    int left = 0;
    int bestLeft = 0;
    int bestLen = Integer.MAX_VALUE;

    for (int right = 0; right < s.length(); right++) {
        char c = s.charAt(right);
        window.put(c, window.getOrDefault(c, 0) + 1);

        if (need.containsKey(c) && window.get(c).intValue() == need.get(c).intValue
()) {
            have++;
        }

        while (have == required) {
            if (right - left + 1 < bestLen) {
                bestLen = right - left + 1;
                bestLeft = left;
            }

            char leftChar = s.charAt(left);
            window.put(leftChar, window.get(leftChar) - 1);
            if (need.containsKey(leftChar) && window.get(leftChar).intValue() < nee
d.get(leftChar).intValue()) {
                have--;
            }
            left++;
        }
    }

    return bestLen == Integer.MAX_VALUE ? "" : s.substring(bestLeft, bestLeft + best
Len);
}
```

Three skills tested in one problem: sliding window mechanics (expand right, shrink left), character frequency tracking with HashMap (Section 5), and shortest-variant update logic (result updated inside the while loop). This is why LeetCode rates it as Hard.

Java-specific note: when comparing Integer objects from a HashMap, use .intValue() or .equals(), not ==. The == operator compares object references, which works for cached values -128 to 127 but fails silently for larger counts. Using .intValue() is always safe.

SLIDE 66B: Minimum Window Substring Step-by-Step Trace

Trace through s = "ADOBECODEBANC", t = "ABC"

Setup: need = {A:1, B:1, C:1}, required = 3.

```
Index:  0  1  2  3  4  5  6  7  8  9  10 11 12
Char:   A  D  O  B  E  C  O  D  E  B  A  N  C
```

Phase 1: Expand until first valid window

The right pointer advances from 0 to 5. At each step, the entering character is added to the window map. Characters A (index 0), B (index 3), and C (index 5) are in the need map and increment "have" when their window count matches the needed count.

right	char	have	Window (relevant chars)	Valid?
0	A	1	A:1	No
3	B	2	A:1, B:1	No
5	C	3	A:1, B:1, C:1	Yes

First valid window: "ADOBEC" (indices 0-5, length 6). Best = 6.

Phase 2: Shrink the first valid window

Shrink from left. Remove s[0]='A'. A's window count drops from 1 to 0, which is below need (1). have drops to 2. Window invalid. left = 1.

Phase 3: Expand to find the second valid window

Right pointer continues from 6 to 10. Characters O, D, E at indices 6-8 are not in need, so have stays at 2. B at index 9 is in need but window already has B:1 which equals need, and adding another makes B:2 which does not trigger a new have increment (only the exact match triggers it). At index 10, A enters. Window A count goes from 0 to 1, matching need. have = 3. Valid.

Window "DOBECODEBA" (indices 1-10, length 10). Worse than current best of 6.

Phase 4: Shrink aggressively

Shrink	Removing	left after	Window Length	have	Best Updated?
s[1]='D'	Not in need	2	9	3	No (9 > 6)
s[2]='O'	Not in need	3	8	3	No (8 > 6)
s[3]='B'	In need, B: 2 to 1, still >= 1	4	7	3	No (7 > 6)
s[4]='E'	Not in need	5	6	3	No (6 = 6, not strictly less)
s[5]='C'	In need, C: 1 to 0, below 1	6	-	2	Invalid, stop

Phase 5: Expand to find the third valid window

Right continues. Index 11 adds N (not in need). Index 12 adds C. Window C goes from 0 to 1, matching need. have = 3. Valid.

Phase 6: Final shrink sequence

Shrink	Removing	left after	Window	Length	Best
(enter while)	-	6	ODEBANC	7	6 (no update)
s[6]='O'	Not in need	7	DEBANC	6	6 (no update)
s[7]='D'	Not in need	8	EBANC	5	5 (updated)
s[8]='E'	Not in need	9	BANC	4	4 (updated)
s[9]='B'	In need, B: 1 to 0, below 1	10	ANC	-	Invalid, stop

Result: "BANC", length 4.

The algorithm found three valid windows (at right = 5, 10, and 12) and aggressively shrank each one. The final shrink sequence discovered the optimal answer by pushing left as far as possible while the window remained valid. This is exactly the shortest-variant behavior: update inside the while loop, keep shrinking as long as the condition holds.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 67: Worked Example: Longest Substring Without Repeating Characters (Revisited)

Problem: Longest Substring Without Repeating Characters (LeetCode 3)

Given a string s, find the length of the longest substring without repeating characters. This problem was introduced in Section 4 (Slide 36). Here we solve it formally using the variable-size window template.

Pattern Recognition

"Longest substring" maps to variable-size sliding window, longest variant. "Without repeating characters" is the condition: every character in the window must be unique. A HashSet tracks the current window's characters.

Mapping to the Template

The template from Slide 65 (longest variant) has three parts: expand (add right element to window state), shrink while condition is violated (remove left element), update result after shrink.

Expand: add s.charAt(right) to the HashSet.
Condition violated: the character at right is already in the set (duplicate found).
Shrink: remove s.charAt(left) from the set, advance left, repeat until the duplicate is gone.
Update: window is valid (all unique), record its length.

```
public int lengthOfLongestSubstring(String s) {
    Set<Character> window = new HashSet<>();
    int left = 0;
    int maxLen = 0;

    for (int right = 0; right < s.length(); right++) {
        while (window.contains(s.charAt(right))) {
```

```
            window.remove(s.charAt(left));
            left++;
        }

        window.add(s.charAt(right));
        maxLen = Math.max(maxLen, right - left + 1);
    }

    return maxLen;
}
```

Trace through "abcabcbb":

right	char	Window before	Duplicate?	Shrink action	Window after	maxLen
0	a	{}	No	-	{a}	1
1	b	{a}	No	-	{a,b}	2
2	c	{a,b}	No	-	{a,b,c}	3
3	a	{a,b,c}	Yes, 'a'	Remove a, left=1	{b,c,a}	3
4	b	{b,c,a}	Yes, 'b'	Remove b, left=2	{c,a,b}	3
5	c	{c,a,b}	Yes, 'c'	Remove c, left=3	{a,b,c}	3
6	b	{a,b,c}	Yes, 'b'	Remove a, left=4; remove b, left=5	{c,b}	3
7	b	{c,b}	Yes, 'b'	Remove c, left=6; remove b, left=7	{b}	3

Result: 3, from substring "abc". Time O(n), space O(min(n, charset size)).

The Template Removes Guesswork

Compare solving this problem from scratch (Section 4) versus recognizing the template (this section). With the template, you follow a mechanical process: identify the pattern signal ("longest substring with condition"), choose the template variant (longest: update after shrink), define the expand/shrink/condition operations (HashSet add/remove/contains), and fill in the template. The solution follows from the structure, not from inventing an approach under pressure.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 68: Wrong vs Right: Sliding Window Mistakes

Mistake 1: Recomputing window state from scratch at every position

Wrong: for each starting position, iterating through the entire window to recompute the sum or frequency count. This gives O(n * K) for fixed-size windows and O(n^2) for variable-size, defeating the entire purpose of

the technique.

Better: maintain the window state incrementally. Add the entering element, remove the leaving element. Each update is O(1), total is O(n).

Mistake 2: Using the longest template for a shortest problem

Wrong: updating the result AFTER the shrink loop when the problem asks for the shortest window. This captures the window at its largest valid state, not its smallest.

Better: identify the optimization direction before writing code. “Longest” or “maximum” means update after the while loop. “Shortest” or “minimum” means update inside the while loop. Drawing the wrong template produces answers that look plausible but fail on edge cases.

Mistake 3: Claiming O(n^2) complexity

Wrong: “The while loop inside the for loop makes this O(n^2).” This misreads the structure. The inner while loop does not run n times per outer iteration.

Better: explain the amortized argument. “The left pointer moves forward at most n times total across all iterations of the outer loop. Combined with the right pointer’s n moves, the total work is at most 2n = O(n).” This is the correct analysis and interviewers expect it.

Mistake 4: Forgetting to update window state when shrinking

Wrong: advancing the left pointer without removing the leaving element from the frequency map, set, or running sum. The window state becomes inconsistent, and subsequent expansions produce wrong results.

Better: every shrink step has two parts. First, remove the leaving element from the window state (subtract from sum, decrement frequency, remove from set). Second, advance left. Missing the first part is one of the most common sliding window bugs.

Mistake 5: Not handling the empty result case

Wrong: returning 0 or a garbage value when no valid window exists. For Minimum Window Substring, if t contains characters not in s, no valid window exists.

Better: initialize the best length to Integer.MAX_VALUE (for shortest) or 0 (for longest). After the loop, check whether the initial value was ever updated. If not, return the appropriate “not found” value (empty string, -1, or 0 depending on the problem).

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 69: Interview Practice: Sliding Window

Practice Prompt 1: Fixed-Size Window

“Find the maximum sum of any contiguous subarray of size K.”

Expected Walkthrough

Step 1, Understand: array of integers (can be negative), K is given, find the maximum sum among all windows of exactly K elements.

Step 2, Pattern: “subarray of size K” is the recognition signal for fixed-size sliding window.

Step 3, Approach: initialize sum of first K elements, slide by adding entering and subtracting leaving element, track maximum.

Step 4, Implement: the solution from Slide 63.

Step 5, Verify: trace through a small example, confirm O(n) time O(1) space.

Practice Prompt 2: Variable-Size Window (Follow-Up)

“Now find the longest subarray with sum less than or equal to a target.”

Expected Walkthrough

Step 1, Understand: the window size is no longer fixed. We want the longest window where the sum does not exceed the target. Array contains positive integers (if negatives were allowed, sliding window would not apply because adding an element could decrease the sum, breaking the monotonic property the shrink relies on).

Step 2, Pattern: “longest subarray with condition” is the signal for variable-size sliding window, longest variant.

Step 3, Approach: expand right to grow the sum, shrink left when sum exceeds target, update result after shrinking.

Step 4, Implement: the solution from Slide 64.

Step 5, Verify: trace, confirm amortized O(n).

Common Fresher Mistakes

Mistake	What Goes Wrong	Fix
Brute force O(n*K) instead of O(n)	Recomputes window from scratch each time	Incremental update: subtract leaving, add entering
Confusing fixed and variable templates	Uses while loop to shrink in a fixed-size problem	Fixed-size has no while loop. The window always moves by exactly one position
Applying sliding window when negatives exist	Shrinking does not guarantee the sum decreases	Sliding window on sums requires positive values. With negatives, use prefix sum + HashMap (Section 3, Slide 25)
Not stating the amortized argument	Says “O(n)” but cannot explain why	Prepare the one-sentence explanation: “left moves at most n times total, so total work is O(n)”

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 70: Section Cheat Sheet

Fixed vs Variable Window Decision

Problem Shape	Window Type	Template Summary
“subarray of size K”, “K consecutive elements”	Fixed-size	Init K elements, slide: add entering, remove leaving
“longest subarray/substring with...”	Variable-size, longest	Expand right, shrink while invalid, update AFTER shrink
“shortest subarray/substring with...”	Variable-size, shortest	Expand right, shrink while VALID (update INSIDE shrink), stop when invalid
“count subarrays with exactly K”	Variable-size, count	atMost(K) - atMost(K - 1), each using the longest template

Pattern Recognition Signals

Signal in Problem Statement	Maps To	Template Variant
“maximum sum of K consecutive”	Fixed sliding window	Fixed-size, track sum
“longest substring without repeating”	Variable window with HashSet	Longest, shrink on duplicate
“smallest window containing all characters”	Variable window with frequency maps	Shortest, shrink while valid
“number of subarrays with at most K distinct”	Variable window with frequency counting	Longest variant as counting helper
“number of subarrays with exactly K distinct”	Two calls to atMost helper	atMost(K) - atMost(K - 1)

Key Complexity Facts

Variant	Time	Space	Prerequisite
Fixed-size	O(n)	O(1) for sums, O(K) for frequency	Window size K given
Variable-size (sum-based)	O(n)	O(1)	All positive values (negatives break monotonicity)
Variable-size (set/map-based)	O(n)	O(min(n, charset))	Condition can be checked/restored in O(1)

Critical Structural Difference: Where to Update the Result

```
LONGEST:   update = Math.max(...)   placed AFTER the while loop
SHORTEST:  update = Math.min(...)   placed INSIDE the while loop

Drawing the wrong one produces subtly wrong answers.
```

The Substring-Subarray Equivalence

“Substring” in a string problem IS “subarray” on the character array. The same fixed-size and variable-size templates apply without modification. When you see “longest substring with condition” or “smallest substring containing X”, you are solving a sliding window problem on characters.

How This Section Connects to Other Sections

Section	Connection
Arrays (Section 3)	Sliding window is one of three core subarray techniques alongside prefix sums and Kadane’s
Strings (Section 4)	Substring problems use the same templates. Character frequency tracking (int[26], HashMap) applies directly
Hash Maps (Section 5)	Variable-size windows frequently need HashMap for tracking frequencies or HashSet for tracking uniqueness
Two Pointers (Section 6)	Sliding window is a specialization of same-direction two pointers with explicit window state management
Stacks (Section 10)	Sliding window maximum uses a monotonic deque. The deque maintains the max efficiently as the window slides

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina